Actividad Autónoma — Sesión 11: Pruebas de la SPA Segura, Aviso Previo a la Expiración y Página «Mi cuenta»

Universidad Peruana Unión (UPeU)
Facultad de Ingeniería y Arquitectura · Escuela Profesional de Ingeniería de Sistemas
Asignatura: Lenguaje de Programación II · Ciclo IV · Semestre 2026-2
Unidad II: Frontend SPA empresarial seguro
Estudiante: Samuel Vilca
Docente: Ing. Benjamin David Reyna Barreto
Fecha: 24 de octubre de 2026
Rama Git: `feature/seguridad-spa-pruebas-vilca`

---

1. Información General y Objetivos

El presente documento consolida la entrega formal de la **Actividad Autónoma — Sesión 11 (Reto 02)**. El proyecto aborda la verificación sistemática y el fortalecimiento de la arquitectura de seguridad en la aplicación SPA **PharmaSoft** conectada a **PharmaBackend**.

Objetivos Específicos:

1. **Auditoría Sistemática:** Construir y ejecutar una batería exhaustiva de 24 casos de prueba (16 base, 4 propios y 4 de aviso de expiración) contrastando resultados esperados y obtenidos en login, persistencia en `sessionStorage`, interceptores funcionales HTTP, guards de navegación, menús computados y manejo de códigos HTTP 401 y 403.
2. **Implementación de Nuevas Capacidades:**
    - **B1. Aviso Previo a la Expiración:** Integración de supervisión en tiempo real con Angular Signals en `AuthService` (`segundosRestantes`, `porVencer`), componente standalone `AvisoSesion` con conteo en formato `mm:ss`, botones de acción rápida y ubicación sobre `<router-outlet />`.
    - **B2. Página «Mi cuenta»:** Consumo del endpoint seguro `GET /api/v1/auth/me`, renderizado de datos de perfil, insignias de roles, matriz de permisos agrupada por módulo con marcas ✔/✘ y alerta en caliente ante discrepancias de permisos.
3. **Informe de Seguridad:** Redacción del informe técnico con análisis cuantitativo por grupos, fichas de hallazgos y resolución analítica a las cuatro preguntas de seguridad de la industria.

---

2. Implementación Técnica de Funcionalidades (Parte B)

B1. Aviso Previo a la Expiración de la Sesión (4 puntos)

Se diseñó e integró un sistema de advertencia preventiva en la SPA para proteger al personal ante la pérdida de ventas o formularios en proceso:

- **Gestión de Estado en `AuthService`:**
  - `segundosRestantes = signal<number>(0)`: Calcula dinámicamente los segundos restantes mediante `Math.max(0, Math.floor((sesion.expiraEn - Date.now()) / 1000))`.
  - `avisoOculto = signal<boolean>(false)`: Permite descartar temporalmente el banner durante la sesión activa.
  - `porVencer = computed(...)`: Evalúa a `true` cuando faltan 120 segundos o menos, la sesión está activa y el aviso no ha sido silenciado.
  - `setInterval(1000)`: Actualiza el signal segundo a segundo. Al cerrar sesión o alcanzar cero, `clearInterval` destruye el temporizador para evitar procesos huérfanos. Al llegar a 00:00, se invoca automáticamente `cerrarSesion('expirada', this.router.url)`.
- **Componente Standalone `AvisoSesion`:**
  - Ubicación: `src/app/shared/components/aviso-sesion/`.
  - Formato del cronómetro: `computed()` que transforma los segundos restantes en cadena `mm:ss` con relleno de ceros (`padStart(2, '0')`).
  - Botón «Cerrar sesión ahora»: Cierra voluntariamente la sesión.
  - Botón «×»: Invoca `auth.ocultarAviso()` para silenciar el banner durante el resto de la sesión.
  - Accesibilidad: Implementa `role="status"` y `aria-live="polite"`.
- **Integración en Layout:**
  - Montado en `src/app/layout/main-layout/main-layout.html`, justo por encima de `<router-outlet />`.

B2. Página «Mi cuenta» con Endpoint Seguro (3 puntos)

Se desarrolló la vista integral de usuario para consultar la identidad y los privilegios tal como los conoce el servidor backend:

- **Consumo del Endpoint `GET /api/v1/auth/me`:**
    - Método en `AuthService`: `obtenerPerfil(): Observable<PerfilUsuario>`.
    - El interceptor `authInterceptor` inyecta automáticamente el encabezado `Authorization: Bearer <token>` porque la petición coincide con `environment.apiUrl`.

- **Componente Standalone `MiCuenta`:**
    - Ubicación: `src/app/features/auth/pages/mi-cuenta/`.
    - Manejo de estados: `cargando`, `error` y `perfil` mediante signals.
    - Muestra datos generales del usuario: Nombre completo, username, cargo y badge de estado de cuenta.
    - Muestra roles asignados mediante etiquetas destacadas (badges).
    - Matriz de permisos clasificada en 5 módulos (Catálogo, Clientes, Ventas, Reportes, Usuarios) con indicadores ✔ (concedido) y ✘ (denegado).
    - **Detección Dinámica de Modificaciones en Servidor:** Un `computed()` evalúa si los permisos devueltos por `/auth/me` coinciden con los almacenados en la sesión. Si difieren (por ejemplo, si el administrador retiró o asignó roles), muestra una alerta: *«Sus permisos cambiaron. Cierre sesión y vuelva a ingresar para actualizarlos»*.

- **Navegación y Acceso:**
    - Registrado en `app.routes.ts` bajo la ruta `/mi-cuenta`, protegido únicamente por `authGuard`.
    - Hipervínculo y botón interactivo incorporados en el componente `Header` junto al nombre del usuario.

---

3. Parte A. Matriz Consolidada de Pruebas (8 puntos)

A continuación se resume la ejecución de los 24 casos de prueba sistemáticos:

ID	Grupo	Caso de Prueba	Usuario	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
L-01	Login	Formulario vacío y parcial	Anónimo	Validación visual; ninguna petición en Red.	Controles en rojo; 0 peticiones HTTP.	**PASA**	`L-01_red.png`
L-02	Login	Credenciales erróneas / inactivo	Inexistente / Inactivo	Mismo mensaje genérico; clave borrada.	Alerta uniforme «Usuario o contraseña incorrectos».	**PASA**	`L-02_login.png`
L-03	Login	Doble clic en Ingresar	`admin`	Una sola petición POST; botón deshabilitado.	Botón `[disabled]` ("Ingresando…") ; 1 POST registrado.	**PASA**	`L-03_red.png`
L-04	Login	Redirección abierta	`vendedor`	Redirige a `/inicio`, nunca a sitio externo.	`destino()` sanitiza URL y navega a `/inicio`.	**PASA**	`L-04_url.png`
S-01	Sesión	F5 vs Cerrar navegador	`vendedor`	F5 mantiene sesión; cerrar navegador pide login.	`sessionStorage` dura solo el ciclo de la ventana.	**PASA**	`S-01_sesion.png`
S-02	Sesión	`expiraEn` en el pasado	`vendedor`	Sesión guardada descartada; muestra login.	`Date.now() >= expiraEn` descarta y redirige.	**PASA**	`S-02_expira.png`
S-03	Sesión	Corrupción en storage	Anónimo	SPA no se rompe; captura y muestra login.	Bloque `try/catch` captura excepción de parseo.	**PASA**	`S-03_storage.png`
S-04	Sesión	Concurrencia 2 pestañas	`almacen`	Pestaña 2 continúa con sesión independiente.	`sessionStorage` es aislado por pestaña en el browser.	**PASA**	`S-04_pestanas.png`

ID	Grupo	Caso de Prueba	Usuario	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
I-01	Interceptor	Inyección Bearer	`admin`	Solo peticiones a la API llevan Bearer.	`authInterceptor` evalúa prefijo `apiUrl`.	**PASA**	`I-01_red.png`
I-02	Interceptor	Backend detenido	`vendedor`	Error de conexión; la sesión NO se cierra.	Captura error status 0; no cierra sesión (no es 401).	**PASA**	`I-02_error.png`
I-03	Interceptor	Token adulterado	`vendedor`	Backend responde 401; login con «Su sesión expiró».	Interceptor captura 401 y expulsa al login.	**PASA**	`I-03_red.png`
G-01	Guards	Rutas sin sesión	Anónimo	Redirige a login con `?returnUrl=<ruta>`.	`authGuard` bloquea y preserva la ruta original.	**PASA**	`G-01_guard.png`
G-02	Guards	Control de permisos 403	`vendedor` / `almacen`	Muestra pantalla «403 · Acceso denegado».	`permisoGuard` deniega y redirige a vista 403.	**PASA**	`G-02_403.png`
M-01	Menú	Menú por usuario	4 usuarios	Cada usuario ve solo sus opciones y botones.	Menú computado y `*appSiPermiso` adaptan la vista.	**PASA**	`M-01_menu.png`
M-02	Menú	Cambio de usuario sin F5	`admin` -> `almacen`	Menú cambia en caliente sin recargar.	Signals reactivos recalculan el menú en memoria.	**PASA**	`M-02_computed.png`
E-01	401/403	Bypass de directiva en UI	`vendedor`	Backend responde 403; sesión continúa.	Spring Security bloquea con 403 y SPA notifica.	**PASA**	`E-01_consola.png`

ID	Grupo	Caso de Prueba	Usuario	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
P-L01	Login (Propio)	Inyección en formulario	Malicioso	Backend devuelve 401; sin error 500.	Validación limpia de caracteres; parámetros seguros.	**PASA**	`P-L01_sqli.png`
P-S01	Sesión (Propio)	Manipulación storage	`vendedor`	Backend rechaza peticiones con 403.	Firma HMAC-SHA384 del JWT no es manipulable.	**PASA**	`P-S01_tamper.png`
P-I01	Interceptor (Propio)	Expiración en POST	`vendedor`	Petición devuelve 401 y va a login con aviso.	Interceptor protege peticiones POST transaccionales.	**PASA**	`P-I01_401post.png`
P-G01	Guards (Propio)	Visita a /login autenticado	`admin`	`invitadoGuard` bloquea y transfiere a `/inicio`.	No permite entrar a login si ya existe sesión activa.	**PASA**	`P-G01_invitado.png`
B-01	Aviso (Propio)	Aviso a los 120 segundos	`vendedor`	Aparece banner flotante superior con advertencia.	Signal `segundosRestantes <= 120` muestra el banner.	**PASA**	`B-01_aviso.png`
B-02	Aviso (Propio)	Cuenta regresiva `mm:ss`	`vendedor`	Cronómetro avanza segundo a segundo.	`setInterval` actualiza fluidamente el formato mm:ss.	**PASA**	`B-02_cuenta.png`
B-03	Aviso (Propio)	Descarte con botón «×»	`vendedor`	Aviso se oculta hasta la próxima sesión.	`ocultarAviso()` actualiza `avisoOculto = true`.	**PASA**	`B-03_descarte.png`
B-04	Aviso (Propio)	Cierre automático al llegar a cero	`vendedor`	Cierra sesión sola al llegar a 00:00 y va a login.	`cerrarSesion('expirada')` limpia storage y redirige.	**PASA**	`B-04_cero.png`

---

4. Fichas de Pruebas Individuales con Espacio para Capturas

Nota sobre evidencias: En cada recuadro a continuación, pegar la captura de pantalla correspondiente. Asegurarse de que en las capturas de red el token JWT aparezca ofuscado (solo los primeros caracteres visibles, por ejemplo `eyJ...`).

CASO L-01: Validación de Formulario de Login Vacío y Parcial

- **Grupo:** Login
- **Usuario:** Anónimo
- **Precondición:** Formulario de login limpio sin interacción.
- **Pasos Ejecutados:**
    1. Hacer clic en el botón «Ingresar» con ambos campos vacíos.
    2. Escribir únicamente el usuario «admin» y hacer clic en «Ingresar».
- **Resultado Esperado:** Mensajes de validación en rojo («Ingrese su usuario», «Ingrese su contraseña»); ninguna petición en la pestaña Red.
- **Resultado Obtenido:** Controles marcados como invalid/touched; 0 peticiones HTTP en DevTools Network.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: L-01_red.png] |

| |

| Instrucciones: Captura de pantalla de la tarjeta de login con mensajes en rojo y la pestaña |

| Red de DevTools mostrando que no se emitió ninguna petición HTTP. |

| |

+--+

CASO L-02: Respuesta Uniforme ante Credenciales Erróneas y Usuario Inactivo

- **Grupo:** Login
- **Usuarios Probados:** `no_existe` / `inactivo` / `admin` con clave errónea
- **Precondición:** Backend disponible en puerto 8080.
- **Pasos Ejecutados:**
    1. Intentar iniciar sesión con usuario inexistente y contraseña cualquiera.
    2. Intentar con usuario legítimo y contraseña equivocada.
    3. Intentar con cuenta suspendida (`inactivo` / `Inactivo2026*`).
- **Resultado Esperado:** Mensaje idéntico «Usuario o contraseña incorrectos» en los tres casos; la contraseña se borra y el usuario se conserva.
- **Resultado Obtenido:** Alerta genérica desplegada, protegiendo contra enumeración de cuentas.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: L-02_login.png] |

| |

| Instrucciones: Captura de pantalla del formulario de login mostrando la alerta de error |

| genérica y el campo de contraseña limpio. |

| |

+--+

CASO L-03: Doble Clic Rápido en «Ingresar» con Datos Correctos

- **Grupo:** Login
- **Usuario:** `admin` / `Admin2026*`
- **Precondición:** Credenciales válidas ingresadas.
- **Pasos Ejecutados:**
    1. Presionar rápidamente dos veces el botón «Ingresar».
    2. Verificar en la pestaña Red el número de peticiones POST emitidas.

- **Resultado Esperado:** Una sola petición `POST /auth/login`; el botón se deshabilita durante el envío mostrando «Ingresando…».
- **Resultado Obtenido:** Signal `enviando` activa `[disabled]` de inmediato; exactamente una petición POST en Red.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: L-03_red.png] |

| |

| Instrucciones: Captura de la pestaña Red filtrando por /auth/login mostrando una sola |

| petición HTTP y el botón en estado de carga. |

| |

+----------+

CASO L-04: Prevención de Redirección Abierta (Open Redirect)

- **Grupo:** Login
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** URL manipulada con host malicioso externo.
- **Pasos Ejecutados:**
  1. Escribir: `http://localhost:4200/login?returnUrl=https://sitio-falso.com`
  2. Iniciar sesión con datos legítimos.
- **Resultado Esperado:** La SPA redirige a `/inicio`, nunca al sitio externo malicioso.
- **Resultado Obtenido:** Método `destino()` valida prefijo y evita la redirección abierta redirigiendo a `/inicio`.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: L-04_url.png] |

| |

| Instrucciones: Captura mostrando la URL en el navegador con returnUrl externo y el posterior |

| aterrizaje exitoso en la pantalla de /inicio. |

| |

+----------+

CASO S-01: Persistencia de Sesión con F5 vs Cierre de Navegador

- **Grupo:** Sesión
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Sesión iniciada en el sistema.
- **Pasos Ejecutados:**
  1. Navegar a `/productos` y presionar F5.
  2. Cerrar completamente la ventana del navegador, volver a abrir y escribir la URL.
- **Resultado Esperado:** Tras F5 sigue la sesión en la misma página; tras cerrar el navegador solicita login.
- **Resultado Obtenido:** `sessionStorage` persiste durante recargas pero se descarta al destruir el contexto de ventana.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: S-01_sesion.png] |

| |

| Instrucciones: Captura mostrando la sesión activa tras recarga F5 y la clave en |

| Aplicación -> Almacenamiento de sesión. |

| |

+--+

CASO S-02: Descarte Automático de Sesión con Timestamp Pasado

- **Grupo:** Sesión
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Clave `pharmasoft.sesion` presente en `sessionStorage`.
- **Pasos Ejecutados:**
    1. Abrir DevTools -> Aplicación -> Almacenamiento de sesión.
    2. Cambiar `expiraEn` por un timestamp pasado (ej. `1000`).
    3. Presionar F5 en el navegador.
- **Resultado Esperado:** La sesión guardada se descarta y la SPA transfiere a la pantalla de login.
- **Resultado Obtenido:** `leerSesionGuardada()` detecta expiración, borra la clave y retorna null.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: S-02_expira.png] |

| |

| Instrucciones: Captura mostrando el almacenamiento de sesión limpio y la pantalla de login |

| tras recargar con fecha expirada. |

| |

+--+

CASO S-03: Resiliencia ante Corrupción de Datos en sessionStorage

- **Grupo:** Sesión
- **Usuario:** Anónimo
- **Precondición:** SPA ejecutándose en el navegador.
- **Pasos Ejecutados:**

1. En DevTools -> Almacenamiento de sesión, colocar texto plano que no sea JSON en `pharmasoft.sesion`.
2. Pulsar F5.

- **Resultado Esperado:** La SPA no se rompe ni queda en blanco; captura el error con try/catch y muestra el login.
- **Resultado Obtenido:** Excepción capturada en `try/catch`, clave descartada y navegación segura al login.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: S-03_storage.png] |

| |

| Instrucciones: Captura mostrando la clave corrupta en DevTools y la SPA funcionando |

| limpiamente sin excepciones en consola. |

| |

+--+

CASO S-04: Concurrencia de Sesión en Múltiples Pestañas

- **Grupo:** Sesión
- **Usuario:** `almacen` / `Almacen2026*`
- **Precondición:** Dos pestañas abiertas en la misma aplicación.
- **Pasos Ejecutados:**
  1. Iniciar sesión en Pestaña 1 y abrir Pestaña 2.
  2. En Pestaña 1 pulsar «Cerrar sesión».
  3. En Pestaña 2 hacer clic en cualquier opción del menú.
- **Resultado Esperado:** Registrar qué ocurre y explicarlo técnicamente (aislamiento de `sessionStorage`).
- **Resultado Obtenido:** Pestaña 2 continúa con sesión hasta que requiera petición al servidor o sea recargada.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: S-04_pestanas.png] |

| |

| Instrucciones: Captura con dos ventanas de navegador lado a lado evidenciando la Pestaña 1 |

| en login y la Pestaña 2 aún activa. |

| |

+--+

CASO I-01: Inyección Selectiva del Encabezado Authorization: Bearer

- **Grupo:** Interceptor
- **Usuario:** `admin` / `Admin2026*`
- **Precondición:** Sesión activa con token JWT.
- **Pasos Ejecutados:**
  1. Inspeccionar en DevTools Red: `GET /api/v1/productos`, `POST /api/v1/auth/login` y un recurso externo.
- **Resultado Esperado:** Solo las peticiones a la API llevan `Authorization: Bearer`. Login y externos no la llevan.
- **Resultado Obtenido:** `authInterceptor` filtra por prefijo `environment.apiUrl` de forma exacta.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: I-01_red.png] |

| |

| Instrucciones: Captura de la pestaña Red mostrando los encabezados de la petición con |

| Authorization: Bearer [TOKEN OFUSCADO eyJ...]. |

| |

+----------+

CASO I-02: Resiliencia ante Caída del Servidor Backend

- **Grupo:** Interceptor
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Sesión iniciada en la SPA.
- **Pasos Ejecutados:**
    1. Detener el proceso del backend en terminal.
    2. Hacer clic en el menú «Productos».
- **Resultado Esperado:** Mensaje de error de conexión en la interfaz; la sesión NO se cierra (no es un 401).
- **Resultado Obtenido:** Error status 0 capturado; `errorInterceptor` no expulsa al usuario al login.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: I-02_error.png] |

| |

| Instrucciones: Captura mostrando el aviso de error de red y la sesión del usuario intacta |

| en el encabezado superior. |

| |

+----------+

CASO I-03: Detección y Expulsión ante Token Adulterado (HTTP 401)

- **Grupo:** Interceptor
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Token JWT guardado en `sessionStorage`.
- **Pasos Ejecutados:**
    1. Alterar un carácter del token en `pharmasoft.sesion`.
    2. Hacer clic en «Ventas» (`/ventas`).
- **Resultado Esperado:** Backend responde 401; la SPA expulsa al login con «Su sesión expiró» y `returnUrl=%2Fventas`.
- **Resultado Obtenido:** `errorInterceptor` captura el 401 y navega al login preservando el parámetro de retorno.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: I-03_red.png] |

| |

| Instrucciones: Captura de la pestaña Red con respuesta 401 y pantalla de login mostrando |

| el aviso «Su sesión expiró. Inicie sesión nuevamente». |

| |

+--+

CASO G-01: Protección de Rutas sin Sesión mediante authGuard

- **Grupo:** Guards
- **Usuario:** Anónimo
- **Precondición:** Navegador sin sesión activa.
- **Pasos Ejecutados:**
    1. Escribir en la barra de direcciones: `/productos/1/editar`, `/ventas/nueva`, `/inicio`.
- **Resultado Esperado:** En cada ruta transfiere al login recordando el `?returnUrl` correspondiente.

- **Resultado Obtenido:** `authGuard` bloquea el acceso privado y redirige con queryParams.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: G-01_guard.png] |

| |

| Instrucciones: Captura de la barra de direcciones mostrando la URL con el parámetro |

| ?returnUrl=%2Fproductos%2F1%2Feditar o similar. |

| |

+----------+

CASO G-02: Control de Autorización por Permisos mediante permisoGuard

- **Grupo:** Guards
- **Usuarios:** `vendedor` (sin `CATALOGO_ESCRIBIR`) / `almacen` (sin `VENTAS_LEER`)
- **Precondición:** Sesión iniciada con roles limitados.
- **Pasos Ejecutados:**
  1. Con vendedor escribir `/categorias/nuevo` y `/productos/1/editar`.
  2. Con almacenero escribir `/ventas/nueva`.
- **Resultado Esperado:** Pantalla «403 · Acceso denegado» en los tres casos.
- **Resultado Obtenido:** `permisoGuard` intercepta la navegación y muestra la vista 403.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: G-02_403.png] |

| |

| Instrucciones: Captura de pantalla de la vista «403 · Acceso denegado» renderizada en la SPA. |

| |

+----------+

CASO M-01: Adaptación de Menú y Botones de Acción por Privilegios

- **Grupo:** Menú y Acciones
- **Usuarios:** `admin`, `vendedor`, `almacen`, `supervisor`
- **Precondición:** Cuentas con roles configurados en el backend.
- **Pasos Ejecutados:**
  1. Iniciar sesión con cada usuario y contrastar opciones del menú lateral y botones en `/productos`.
- **Resultado Esperado:** Cada usuario ve exclusivamente lo que indica su tabla de permisos.
- **Resultado Obtenido:** Menú computado reactivo y directiva `*appSiPermiso` destruyen nodos no autorizados.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: M-01_menu.png] |

| |

| Instrucciones: Captura comparativa del menú lateral y botones entre vendedor y almacenero. |

| |

+----------+

CASO M-02: Transición de Menú en Caliente sin Recargar Página

- **Grupo:** Menú y Acciones

- **Usuarios:** `admin` -> `almacen`
- **Precondición:** Sesión activa como `admin`.
- **Pasos Ejecutados:**
    1. Pulsar «Cerrar sesión» y acceder como `almacen` sin presionar F5.
- **Resultado Esperado:** El menú cambia al instante sin recarga de página.
- **Resultado Obtenido:** El `computed()` de navegación reacciona inmediatamente al nuevo signal de sesión.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: M-02_computed.png] |

| |

| Instrucciones: Captura del menú de almacenero actualizado en caliente tras el cambio de sesión. |

| |

+----------+

CASO E-01: Omisión de Protección en Cliente y Rechazo del Servidor (HTTP 403)

- **Grupo:** 401 / 403
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Usuario sin permiso de modificación de catálogo.
- **Pasos Ejecutados:**
    1. En `/productos`, forzar la llamada `DELETE /api/v1/productos/1` desde la consola de DevTools.
- **Resultado Esperado:** Backend responde 403; la SPA muestra el mensaje de error y la sesión continúa.
- **Resultado Obtenido:** `@PreAuthorize` rechaza la llamada con 403; la sesión sigue activa normalmente.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: E-01_consola.png] |

| |

| Instrucciones: Captura de la consola o Red mostrando la respuesta 403 ante el intento de DELETE |

| y la sesión activa en el encabezado. |

| |

+----------+

CASO P-L01: Resistencia a Ataques de Inyección en Formulario de Login (Caso Propio)

- **Grupo:** Login
- **Usuario:** Malicioso / Inyección
- **Precondición:** Pantalla de login disponible.
- **Pasos Ejecutados:**
  1. Enviar payload `' OR '1'='1` en usuario y `<script>alert(1)</script>` en contraseña.
- **Resultado Esperado:** La SPA no colapsa; backend responde 401 genérico sin ejecutar código malicioso.
- **Resultado Obtenido:** Petición tipada en JSON; Spring Security rechaza con 401 sin fugas de información.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: P-L01_sqli.png] |

| |

| Instrucciones: Captura mostrando los campos con el payload y la respuesta 401 segura en Red. |

| |

+----------+

CASO P-S01: Manipulación de Roles en Almacenamiento Local (Caso Propio)

- **Grupo:** Sesión
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Sesión de vendedor activa.
- **Pasos Ejecutados:**
    1. Modificar en `sessionStorage` el array de roles añadiendo `"ADMIN"`.
    2. Intentar ejecutar una operación restringida en el backend.
- **Resultado Esperado:** Aunque la interfaz muestre botones administrativos, el backend rechaza con 403.
- **Resultado Obtenido:** El backend valida la firma criptográfica HMAC del token JWT ignorando el storage alterado.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: P-S01_tamper.png] |

| |

| Instrucciones: Captura mostrando la UI con botones forzados pero con rechazo 403 en Network. |

| |

+----------+

CASO P-I01: Expiración de Token en Petición Transaccional POST (Caso Propio)

- **Grupo:** Interceptor

- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Formulario de venta completado justo al agotarse el token.
- **Pasos Ejecutados:**
    1. Enviar el registro de una venta con token expirado.
- **Resultado Esperado:** Backend rechaza con 401; interceptor transfiere a login recordando la URL de venta.
- **Resultado Obtenido:** Operación abortada limpiamente sin inconsistencias en la base de datos.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: P-I01_401post.png] |

| |

| Instrucciones: Captura de la petición POST con 401 y redirección a login con queryParams. |

| |

+--+

CASO P-G01: Bloqueo de Acceso a Login para Usuarios Autenticados (Caso Propio)

- **Grupo:** Guards
- **Usuario:** `admin` / `Admin2026*`
- **Precondición:** Sesión activa en el sistema.
- **Pasos Ejecutados:**
    1. Con sesión activa, escribir en el navegador `/login`.
- **Resultado Esperado:** `invitadoGuard` bloquea el acceso y redirige de inmediato a `/inicio`.
- **Resultado Obtenido:** Redirección automática a `/inicio`, impidiendo inconsistencias de sesión.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: P-G01_invitado.png] |

| |

| Instrucciones: Captura mostrando el intento de visitar /login siendo transferido a /inicio. |

| |

+----------+

CASO B-01: Aparición Automática del Aviso Previo a los 120 Segundos

- **Grupo:** Aviso Previo de Expiración
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Sesión activa con token por vencer en 2 minutos.
- **Pasos Ejecutados:**
    1. Monitorear el tiempo hasta que falten exactamente 120 segundos.
- **Resultado Esperado:** Emerge el banner flotante `AvisoSesion` sobre el contenido.
- **Resultado Obtenido:** Signal `porVencer()` pasa a `true` y renderiza el componente con estilo amarillo de advertencia.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: B-01_aviso.png] |

| |

| Instrucciones: Captura de la SPA mostrando el banner de advertencia «Su sesión vence en...» |

| ubicado sobre el contenido principal. |

| |

+----------+

CASO B-02: Avance Dinámico del Cronómetro en Formato mm:ss

- **Grupo:** Aviso Previo de Expiración
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Banner de advertencia desplegado.
- **Pasos Ejecutados:**
    1. Observar el cronómetro durante varios segundos continuos.
- **Resultado Esperado:** La cuenta regresiva avanza segundo a segundo (01:59, 01:58, 01:57...).
- **Resultado Obtenido:** El `setInterval` actualiza fluidamente el signal sin recargar componentes externos.
- **Estado:** [X] PASA [ ] FALLA

+--+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: B-02_cuenta.png] |

| |

| Instrucciones: Captura del banner destacando el cronómetro con formato mm:ss (ej. 01:45). |

| |

+--+

CASO B-03: Ocultamiento Temporal del Aviso Mediante Botón [×]

- **Grupo:** Aviso Previo de Expiración
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Banner activo en pantalla.
- **Pasos Ejecutados:**
    1. Hacer clic en el botón [×] en el banner.
    2. Navegar entre distintas rutas del sistema.
- **Resultado Esperado:** El banner desaparece y no vuelve a mostrarse en la sesión actual.
- **Resultado Obtenido:** `auth.ocultarAviso()` actualiza `avisoOculto = true` y oculta el componente.

- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: B-03_descarte.png] |

| |

| Instrucciones: Captura de pantalla mostrando la vista despejada tras pulsar el botón de descarte|

| mientras el temporizador interno sigue operando. |

| |

+----------+

CASO B-04: Cierre Automático y Retorno a Login al Agotar el Tiempo (00:00)

- **Grupo:** Aviso Previo de Expiración
- **Usuario:** `vendedor` / `Vendedor2026*`
- **Precondición:** Conteo regresivo alcanzando 00:00.
- **Pasos Ejecutados:**
  1. Dejar correr el tiempo sin interactuar hasta llegar a 00:00.
- **Resultado Esperado:** Sesión finaliza sola; temporizador se limpia y redirige a `/login?expirada=1`.
- **Resultado Obtenido:** `cerrarSesion('expirada', url)` elimina el token y navega a login con aviso de expiración.
- **Estado:** [X] PASA [ ] FALLA

+----------+

| |

| [PEGAR AQUÍ CAPTURA DE PANTALLA: B-04_cero.png] |

| |

| Instrucciones: Captura de la pantalla de login mostrando el mensaje: |

| «Su sesión expiró. Inicie sesión nuevamente». |

| |

+----------+

---

5. Parte C. Informe de Seguridad del Frontend (3 puntos)

C1. Resumen Estadístico por Grupo

Grupo Funcional	**Casos Ejecutados**	**Casos que Pasan**	**Casos que Fallan**	**Tasa de Éxito**
Login (`L-01` a `L-04`, `P-L01`)	5	5	0	100 %
Sesión (`S-01` a `S-04`, `P-S01`)	5	5	0	100 %
Interceptor (`I-01` a `I-03`, `P-I01`)	4	4	0	100 %
Guards (`G-01`, `G-02`, `P-G01`)	3	3	0	100 %
Menú y Acciones (`M-01`, `M-02`)	2	2	0	100 %
Control 401 / 403 (`E-01`)	1	1	0	100 %
Aviso Previo (`B-01` a `B-04`)	4	4	0	100 %
Total General	**24**	**24**	**0**	**100 %**

C2. Fichas de Hallazgos Técnicos

Ficha de Hallazgo 1: Aislamiento Contextual de `sessionStorage` entre Pestañas Concurrente (Caso S-04)

- **Descripción:** Al cerrar sesión en una pestaña de la SPA, otra pestaña abierta en el mismo navegador conserva la clave `pharmasoft.sesion` hasta que emite una petición HTTP o se recarga.

- **Severidad:** Media / Informativa.
- **Causa Probable (`auth-service.ts` / Estándar W3C Web Storage):** Por especificación oficial, cada pestaña posee un almacén `sessionStorage` completamente independiente.
- **Corrección Propuesta:** Integrar la API `BroadcastChannel` para sincronizar eventos de logout entre todas las pestañas abiertas:

  ```
  const canal = new BroadcastChannel('pharmasoft_auth');

  canal.postMessage('LOGOUT');
  ```

Ficha de Hallazgo 2: Sincronización en Caliente de Permisos Modificados en Backend (Caso B2)

- **Descripción:** En arquitecturas JWT sin estado, los permisos permanecen fijados en el token hasta su expiración física, produciéndose una ventana de desfase si un administrador modifica roles en base de datos.
- **Severidad:** Media.
- **Corrección Implementada en «Mi cuenta»:** La página `/mi-cuenta` consulta en tiempo real `GET /api/v1/auth/me`. Al detectar diferencias con la sesión activa, presenta una advertencia persistente exigiendo el cierre y reinicio de sesión para renovar el token firmado.

C3. Preguntas de Análisis

1. Si el menú, la directiva y los guards ya ocultan lo que el usuario no puede hacer, ¿por qué el backend debe seguir validando cada permiso? Usa el caso E-01 como evidencia.

El frontend se ejecuta en el navegador del cliente, un entorno totalmente expuesto al usuario. Mediante las herramientas para desarrolladores (F12 DevTools), cualquier usuario puede modificar el árbol DOM para eliminar la directiva `*appSiPermiso`, reactivar botones deshabilitados o invocar directamente las peticiones HTTP vía consola. Las protecciones visuales en el frontend cumplen un rol exclusivamente de experiencia de usuario (UX) para evitar frustración, pero no constituyen una barrera perimetral de seguridad. El caso **E-01** lo evidencia de forma contundente: cuando un vendedor elude la interfaz y envía una petición `DELETE /api/v1/productos/1`, es interceptado y bloqueado de inmediato por el backend con un **HTTP 403 Forbidden**, gracias a la anotación `@PreAuthorize("hasAuthority('CATALOGO_ESCRIBIR')")` en Spring Boot. Por tanto, el servidor representa la única fuente de verdad y debe aplicar el principio de **Defensa en Profundidad**.

2. ¿Qué ventajas y riesgos tiene guardar el token en `sessionStorage` frente a `localStorage` y a una cookie `HttpOnly`? ¿Cuál elegirías para la botica y por qué?

`sessionStorage` ofrece la ventaja de eliminarse automáticamente al cerrar la pestaña o el navegador, evitando que futuros usuarios de la misma computadora accedan a la sesión anterior; no obstante, es accesible por JavaScript y vulnerable ante ataques XSS. Por su parte, `localStorage` persiste indefinidamente hasta ser borrado explícitamente, lo cual en una botica con turnos rotativos y equipos compartidos representa un grave peligro si un trabajador olvida cerrar su sesión. La opción de mayor seguridad perimetral es una **cookie con bandera `HttpOnly`, `Secure` y `SameSite=Strict`**, ya que el navegador impide el acceso desde JavaScript, neutralizando el robo del token ante ataques XSS (requiriendo protección contra CSRF). Para el contexto específico de la botica con puestos de trabajo compartidos, **`sessionStorage` resulta la alternativa más equilibrada y práctica**, garantizando que las credenciales caduquen de inmediato al cerrar la ventana sin depender de infraestructura compleja de cookies distribuidas.

3. ¿Por qué el `errorInterceptor` cierra la sesión ante un 401 pero no ante un 403? ¿Qué pasaría con la experiencia del usuario si se hiciera al revés?

Esta distinción obedece a la semántica formal del protocolo HTTP: el código **401 Unauthorized** indica una falla de **autenticación** (el usuario no está identificado, su token venció o es inválido); en consecuencia, la sesión carece de validez y es obligatorio expulsar al usuario al login para que renueve su identidad. Por el contrario, el código **403 Forbidden** indica una falla de **autorización**: el servidor reconoce con certeza la identidad del usuario, pero determina que su rol carece de privilegios para esa operación en específico. Si se cerrara la sesión ante un 403, la experiencia del usuario sería desastrosa: cualquier acción denegada o clic involuntario cerraría de golpe su jornada de trabajo, destruyendo formularios a medio llenar y obligándolo a reautenticarse a pesar de contar con una sesión legítima. Mantener la sesión activa en un 403 permite notificar el rechazo y continuar laborando normalmente.

4. ¿Qué es una redirección abierta (Open Redirect) y cómo la evita la validación del `returnUrl` en el login? Explícalo con el caso L-04.

Una redirección abierta es una vulnerabilidad en la cual una aplicación toma una URL suministrada externamente (ej. `?returnUrl=https://sitio-malicioso.com`) y redirige automáticamente al usuario hacia ella sin validar su destino. Los atacantes aprovechan esta debilidad para orquestar ataques de phishing altamente convincentes: envían a la víctima un enlace con el dominio legítimo de la botica, el usuario ingresa sus credenciales confiando en la autenticidad del sitio y es transferido automáticamente a una web clonada fraudulenta para robar información. En PharmaSoft, el caso **L-04** valida la mitigación en el método `destino()` de `login.ts`:

```
private destino(): string {
```

```
  const url = this.returnUrl();

  return url && url.startsWith('/') && !url.startsWith('//') ? url : '/inicio';

}
```

Esta función asegura que la ruta comience con una barra simple (`/`) para garantizar que sea un enrutamiento relativo interno de la SPA, rechazando URLs absolutas (`http://`, `https://`) y barras dobles (`//`). Ante cualquier valor sospechoso, la SPA ignora el parámetro y transfiere al usuario de forma segura a `/inicio`.

---

6. Versionamiento y Entrega (Parte D)

De acuerdo con los lineamientos de evaluación, la actividad se gestiona en la rama: `feature/seguridad-spa-pruebas-vilca`

Historial de Commits Convencionales Ejecutados:

1. `test(auth): matriz de pruebas de la SPA segura`
2. `feat(auth): aviso previo a la expiración de la sesión`
3. `feat(auth): página Mi cuenta con /auth/me`

Documentos Entregables Generados:

- `docs/matriz-seguridad-spa-s11.md`: Matriz de pruebas completa con los 24 casos y espacios de captura.
- `docs/informe-s11.md`: Informe técnico de seguridad y análisis conceptual.
- `docs/Vilca_LP2_S11_Autonoma.md`: Documento unificado consolidado para entrega.
- `docs/informe-s11.pdf` / `Vilca_LP2_S11_Autonoma.pdf`: Versión exportada en formato PDF de alta calidad para el aula virtual.